

Föreläsning 24

Ändrad 20 Oct 10:15

Separat kompilering av C-filer

I medelstora och stora programprojekt delar man alltid upp programmets funktioner i olika filer. Och kompilerar dessa filer var för sig.

Därefter "länkar" man ihop de kompillerade filerna till ett färdigt program.

Denna möjlighet är omistlig när man skriver större program. Man vill inte gärna kompilera om en miljon rader kod bara för att man ändrat litet grand i en liten funktion någonstans.

I C är detta tämligen enkelt. På mitt system (*gcc* under *Unix*) säger man åt kompilatorn att låta bli att skapa ett körbart program. Istället skapar kompilatorn en fil där de kompillerade funktionerna ligger. Denna fil har vanligen vanligen tillägget ".o" (innehållet kallas "objektkod").

Därefter kan man ha ".o"-filen som parameter till kompilatorn när man kompilerar sina C-program. C-programmet kan då referera till de kompillerade funktionerna i ".o"-filen.

För att kompilatorn ska kunna göra sitt jobb ordentligt behöver den dock veta vissa saker om funktionerna. Det är här det ständigt återkommande `#include <stdio.h>` kommer in i bilden.

Programmeraren skapar en egen ".h"-fil ("header-fil") för varje ".c"-fil ("källkodsfil") han vill kompilera separat.

Header-filen ska innehålla funktionshuvudet för varje funktion som ska kunna användas av andra program.

■ Exempel: Vi utgår från min miniräknare från [förrförra föreläsningen](#). Men vi klipper bort funktionen `main()` från filen (för att datorn inte ska bli förvirrad). Filen heter "calc.c".

Jag vill nu *exportera* funktionerna `calculator()`, `printtime()` och `evaluate_expression()` från "calc.c". Därför skapar jag en ny fil vid namn "calc.h" med följande innehåll:

```
1 extern void calculator();
2 extern void printtime();
3 extern double evaluate_expression(double, char, double);
```

Lägg märke till nyckelordet `extern` som ska in framför funktionsnamnen.

Lägg också märke till hur vi anger parametrarna till den funktion som har parametrar: vi anger endast parametrarnas typer, inte deras namn. Typerna är det enda kompilatorn behöver veta. Det är för att kompilatorn ska kunna

kontrollera att anrop är korrekta.

Detta betyder inte nödvändigtvis att kompilatorn faktiskt kontrollerar att anrop är korrekta.

Bara att den kan göra det om den har lust. Du kan inte slappna av förrän du är säker på att du vet hur noga din just kompilator kollar anropen.

Därefter kompilerar jag filen "calc.c", och talar särskilt om för kompilatorn att jag bara är intresserad av att få en objektfil, och ingen programfil. Datorn skapar då filen "calc.o".

Nu ska jag skapa en tredje fil med en funktion `main()` -- för att kolla att allt fungerar. Denna fil kallar jag "test.c":

```
1 #include <stdio.h>
2 #include "calculator.h"
3
4 extern void printf();
5
6 void main()
7 {
8     printtime();
9     printf("%g\n", evaluate_expression(12, '/', 4));
10    calculator();
11    printtime();
12 }
```

Lägg märke till include-satsen på rad 2.

Slutligen kompilerar jag "test.c", och instruerar -- på något systemberoende sätt -- kompilatorn att leta i "calc.o" efter färdigkompileerade funktioner.

Detta ger mig ett körbart program, som ger följande utskrift:

```
1 Time is 01:03
2 3
3 = 0
4 > 2 + 2
5 = 4
6 > quit
7 bye
8 Time is 01:03
```

Globala variabler

Du kan deklarera en variabel på ett sådant sätt att flera funktioner kan komma åt den, och ändra i den. Men gör helst inte det!

Det normala sättet att låta flera funktioner manipulera samma variabel är att de skickar variabeln som pekarparameter mellan sig.

Men om du nödvändigtvis måste, så behöver du bara deklarera variabeln "ytterst" -- ute bland funktionerna. Då kan variabeln användas av alla funktioner som är deklarerade efter variabeln.

Bilting/&Skansholm ger några exempel på legitima användningsområden för *globala variabler* -- som de kallas -- i kapitel 6.5.3.

Men glöm inte möjligheten att deklarera en variabel

`static` -- det är kanske det du egentligen är ute efter.

Du kan exportera en global variabel, precis som du exporterar funktioner, genom att lägga till

```
extern double GlobalVariabel;
```

i ".h"-filen.

Funktioner som värden

Funktioner kan användas som värden -- variabler kan ha funktioner som värden -- eller mer exakt *adresser* till funktioner kan användas som värden.

■ Exempel, syntaxen är bara att acceptera:

```
double (*f)(double);
```

Variabeln `f` kan nu ges adressen till en funktion som värde. Men bara till en funktion av rätt typ: funktionen ska ta en `double`-parameter och returnera en `double`. Vi kan alltså skriva:

```
f = &sin;
x = (*f)(3.14); /* x = sin(3.14) */
f = &cos;
x = (*f)(3.14); /* x = cos(3.14) */
```

Variabeln `f` är en funktionspekare. Men adressoperatoren kan utelämnas (i alla fall på mitt system)

```
f = sin;
```

Dessutom kan jag på mitt system skriva `f(3.14)` i stället för `(*f)(3.14)`. Men det är enligt Bitling/Skansholm ganska ovanligt.

■ Större exempel: jag har gjort en ny implementation av min funktion `evaluate_function()` från [förrförra föreläsningen](#).

Flera rader liknande varandra, i `if`-stegen. Jag lade in det lilla som varierade i en array. Det som varierade var namnet på funktionen, och funktionen själv:

```
1 double sqr(double x)
2 {
3     return x * x;
4 }
5
6
7 double cube(double x)
8 {
9     return x * x * x;
10 }
11
12
13 typedef struct {
14     char name[5];
15     double (*f)();
16 } function;
17
18
```

```

19 typedef struct {
20     unsigned short size;
21     function f[30];
22 } functions;
23
24
25 /* Ex: evaluate_function("sin", 1.3) == sin(1.3)
26 */
27 double evaluate_function(char fname[], double x)
28 {
29     static functions F = {7, {"sin", sin}, {"cos", cos},
30                             {"tan", tan}, {"ln", log},
31                             {"sqrt", sqrt}, {"sqr", sqr},
32                             {"cube", cube}};
33     unsigned short i = 0;
34
35     while (i < F.size && strcmp(F.f[i].name, fname) != 0)
36         i++;
37
38     if (strcmp(F.f[i].name, fname) == 0)
39         x = (*(F.f[i].f))(x);
40     else
41         fprintf(stderr, "Unknown funktion: %s\n", fname);
42     return x;
43 }

```

Jag använder mig av funktioner som värden. Jag har definierat en `struct` som heter `function` vars ena fält `f` är en funktion, och det andra fältet `name` en sträng (rad 13--16).

Dessutom har jag definierat en `struct` vid namn `functions` (obs plural) som lagrar en `function`-array, plus `size` för att hålla reda på antalet element i arrayen. (rad 19--22).

I `evaluate_function()` deklarerar jag en variabel `F` av typen `functions`. Därefter initierar jag `F` med lämpliga värden (rad 29--32).

Variabeln `F` är deklarerad `static` -- det betyder att den behåller sitt värde mellan anropen -- det betyder att initieringen bara utförs en enda gång.

De flesta funktioner jag använder finns i `math.h`. Två funktioner har jag dock skrivit själv: `cube` och `sqr`.

Min algoritim: jag söker igenom arrayen `F.f` i jakt på ett funktionsnamn som matchar `fname`. Om jag hittar detta funktionsnamn, applicerar jag funktionen (rad 39). Annars skriver jag ett felmeddelande.

Den nya implementationen av funktionen fungerar ihop med resten av programmet. Jag har testat.

Samma lektion igen

